UNIT – I

Python Programming:

Introduction:

- According to Wikipedia,
  - Computer programming is the process of designing and building an executable computer program to accomplish a specific computing result or to perform a specific task.
- Programming involves tasks such as:
  - Analysis, generating algorithms, profiling algorithms accuracy and resource consumption, and the implementation of algorithms in a chosen programming language (commonly referred to as coding)".
- In a nutshell, computer programming, or coding, as it is sometimes known, is telling a computer to do something using a language it understands.
- A proper introduction:
  - We love to make references to the real world when we teach coding; we believe they help people to better retain the concepts.
  - However, now is the time to be a bit more difficult and see what coding is from a more technical perspective.
  - If you write a piece of software that allows people to buy clothes online, you will have to represent real people, real clothes, real brands, sizes, and so on and so forth, within the boundaries of a program.
  - In order to do so, you will need to create and handle objects in the program being written. A person or A car is an object, Luckily, Python understands objects very well.
- The two main features any object has are properties and methods.
- Let's take the example of a person as an object. Typically, in a computer program, you'll represent people as customers or employees.
  - The properties that you store against them are things like a name, a Social Security number, an age, whether they have a DL, an email, gender, and so on.
  - So, properties are characteristics of an object.
- Methods are things that an object can do.
  - As a person, I have methods such as speak, walk, sleep, wake up, eat, dream, write, read, and so on.
  - All the things that I can do could be seen as methods of the objects that represent me.

- Objects are Python's abstraction for data. All data in a Python program is represented by objects or by relations between objects.
- Python is the marvelous creation of Guido Van Rossum, a Dutch computer scientist and mathematician who decided to gift the world with a project he was playing around with over Christmas 1989.
- The language appeared to the public somewhere around 1991, and since then has evolved to be one of the leading programming languages.
- Python represents the following qualities:

a. Portability:

- Python runs everywhere, and porting a program from Linux to Windows or Mac is usually just a matter of fixing paths and settings.
- Python is designed for portability and it takes care of specific operating system (OS) quirks behind interfaces that shield you from the pain of having to write code tailored to a specific platform.

b. Coherence:

- Python is extremely logical and coherent. Most of the time you can just guess how a method is called if you don't know it.
- It means less cluttering in your head, as well as less skimming through the documentation, and less need for mappings in your brain when you code.

c. Developer productivity:

- According to Mark Lutz, a Python program is typically one-fifth to one-third the size of equivalent Java or C++ code. This means the job gets done faster.
- And faster is good. Faster means being able to respond more quickly to the market. Less code not only means less code to write, but also less code to read, maintain, debug, and refactor.
- Another important aspect is that Python runs without the need for lengthy and time-consuming compilation and linkage steps.

d. An extensive library:

- Python has an incredibly extensive standard library.
- If that wasn't enough, the Python international community maintains a body of third-party libraries, tailored to specific needs, available in Python Package Index (PyPI).

e. Software quality:

- Python is heavily focused on readability, coherence, and quality. The language’s uniformity allows for high readability as coding is more of a collective effort than a solo endeavor.

- Another important aspect of Python is its intrinsic multiparadigm nature.
- You can use it as a scripting language, but you can also exploit object-oriented, imperative, and functional programming styles—it is extremely versatile.

f. Software integration:

- Another important aspect is that Python can be extended and integrated with many other languages.
- Which means that even when a company is using a different language as their mainstream tool, Python can come in and act as a gluing agent between them.

❖ **What are the drawbacks?**

- Probably, the only drawback that one could find in Python, which is not due to personal preferences, is its execution speed.
- Typically, Python is slower than its compiled siblings.
- The standard implementation of Python produces, when you run an application, a compiled version of the source code called byte code (with the extension .pyc), which is then run by the Python interpreter.
- The advantage of this approach is portability, which we pay for with increased runtimes due to the fact that Python is not compiled down to the machine level, as other languages are.

Python's execution model:

- It comprises of some important concepts, such as scope, names, and namespaces.

Names and namespaces:

- Say you are looking for a book, so you go to the library and ask someone to obtain this. They tell you something like Second Floor, Section X, Row Three.
- It would be very different to enter a library where all the books are piled together in random order in one big room. No floors, no sections, no rows, no order. Fetching a book would be extremely hard.
- When we write code, we have the same issue: we have to try and organize it so that it will be easy for someone who has no prior knowledge about it to find.
- When software is structured correctly, it also promotes code reuse.
- As a first example, let us take a book. We refer to a book by its title; in Python lingo, that would be a name.

- Python names are the closest abstraction to what other languages call variables. Names basically refer to objects and are introduced by name-binding operations.
- Let's see a quick example (again, notice that anything that follows a # is a comment):
- Remember that each Python object has an identity, a type, and a value.
- We defined three objects in the preceding code; let's now examine their types and values……
  - An integer number n (type: int, value: 3)
  - A string address (type: str, value: Sherlock Holmes' address)
  - A dictionary employee (type: dict, value: a dictionary object with three key/value pairs).

```
>>> n = 3  # integer number
>>> address = "221b Baker Street, NW1 6XE, London"  # Sherlock Holmes'
address
>>> employee = {
...     'age': 45,
...     'role': 'CTO',
...     'SSN': 'AB1234567',
... }
>>> # let's print them
>>> n
3
>>> address
'221b Baker Street, NW1 6XE, London'
>>> employee
{'age': 45, 'role': 'CTO', 'SSN': 'AB1234567'}
>>> other_name
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'other_name' is not defined
>>>
```

- So, what are n, address, and employee? They are **names**, and these can be used to retrieve data from within our code.
- They need to be kept somewhere so that whenever we need to retrieve those objects, we can use their names to fetch them.
- We need some space to hold them, hence: **namespaces**. A namespace is a mapping from names to objects. Examples are……
  - The set of built-in names (containing functions that are always accessible in any Python program).
  - The global names in a module, and the local names in a function.
  - Even the set of attributes of an object can be considered a namespace.

- The beauty of namespaces is that they allow you to define and organize your names with clarity, without overlapping or interference.
- For example, the namespace associated with the book we were looking for in the library can be used to import the book itself, like this……

  from library.second_floor.section_x.row_three import book

- We start from the library namespace, and by means of the dot (.) operator, we walk into that namespace.

Scopes:

- According to Python's documentation:
  - "A scope is a textual region of a Python program, where a namespace is directly accessible."
- Directly accessible means that, when looking for an unqualified reference to a name, Python tries to find it in the namespace.
- Scopes are determined statically, but actually, during runtime, they are used dynamically. This means that by inspecting the source code, you can tell what the scope of an object is.
- There are four different scopes that Python makes accessible……
  - The local scope, which is the innermost one and contains the local names.
  - The enclosing scope; that is, the scope of any enclosing function. It contains non-local names and also non-global names.
  - The global scope contains the global names.
  - The built-in scope contains the built-in names. Python comes with a set of functions that you can use in an off-the-shelf fashion, such as print, all, abs, and so on. They live in the built-in scope.
- The rule is the following:
  - When we refer to a name, Python starts looking for it in the current namespace.
  - If the name is not found, Python continues the search in the enclosing scope, and this continues until the built-in scope is searched.
  - If a name has still not been found after searching the built-in scope, then Python raises a NameError exception, which basically means that the name hasn't been defined.
- The order in which the namespaces are scanned when looking for a name is therefore **local, enclosing, global, built-in (LEGB).**

Objects and classes:

- When we introduced objects previously in the A proper introduction section of the chapter, we said that we use them to represent real-life objects.
- We have already seen that objects are Python's abstraction for data.
- In fact, everything in Python is an object: numbers, strings (data structures that hold text), containers, collections, even functions.
- You can think of them as if they were boxes with at least three features: an ID (which is unique), a type, and a value.
- But how do they come to life? How do we create them? How do we write our own custom objects? The answer lies in one simple word: classes.
- Objects are, in fact, instances of classes. For now, the best way for you to get the difference between classes and objects is by means of an example……
- Say a friend tells you, I bought a new bike! You immediately understand what she's talking about.
  - Have you seen the bike? No.
  - Do you know what color it is? Nope. The brand? Nope.
  - Do you know anything about it? Nope.
- But at the same time, you know everything you need in order to understand what your friend meant when she told you that she bought a new bike.
- In computer programming, that is called a class. It's that simple. Classes are used to create objects.
- In other words, we all know what a bike is; we know the class.
- But then your friend has her own bike, which is an instance of the bike class. Her bike is an object with its own characteristics and methods.
- You have your own bike. Same class, but different instance. Every bike ever created in the world is an instance of the bike class.
- So many interesting things to notice here.
  - First, the definition of a class happens with the class statement.
  - Whatever code comes after the class statement, and is indented, is called the body of the class.
  - In our case, the last line that belongs to the class definition is print("Braking!").
  - After having defined the class, we are ready to create some instances.

```python
# bike.py
# let's define the class Bike
class Bike:

    def __init__(self, colour, frame_material):
        self.colour = colour
        self.frame_material = frame_material

    def brake(self):
        print("Braking!")

# let's create a couple of instances
red_bike = Bike('Red', 'Carbon fiber')
blue_bike = Bike('Blue', 'Steel')

# let's inspect the objects we have, instances of the Bike class.
print(red_bike.colour)  # prints: Red
print(red_bike.frame_material)  # prints: Carbon fiber
print(blue_bike.colour)  # prints: Blue
print(blue_bike.frame_material)  # prints: Steel
```

- You can see that the class body hosts the definition of two methods. A method is basically a function that belongs to a class.
- The first method, __init__, is an initializer. It uses some Python magic to set up the objects with the values we pass when we create it.
- Every method that has leading and trailing double underscores, in Python, is called a magic method. Magic methods are used by Python for a multitude of different purposes.
- The other method we defined, brake, is just an example of an additional method that we could call if we wanted to brake. It contains only a print statement.
- One last thing to notice: remember how we said that the set of attributes of an object is considered to be a namespace?
- See that by getting to the frame_type property through different namespaces (red_bike, blue_bike), we obtain different values. No overlapping, no confusion.
- The dot (.) operator is of course the means we use to walk into a namespace, in the case of objects as well.

Built-In Data Types:

- Everything you do with a computer is managing data. Data comes in many different shapes and flavors.
  - It's the music you listen to, the movies you stream, the PDFs you open. Even the source of the chapter you're reading at this very moment is just a file, which is data.
  - Data can be simple, whether it is an integer number to represent an age, or complex, like an order placed on a website.

- It can be about a single object or about a collection of them.
- Data can even be about data—that is, metadata. This is data that describes the design of other data structures, or data that describes application data or its context.

- In Python, objects are our abstraction for data.
- Python has an amazing variety of data structures that you can use to represent data or combine them to create your own custom data.

Everything is an object:

- Before we delve into the specifics, we want you to be very clear about objects in Python, so let's talk a little bit more about them.
- As we already said, everything in Python is an object.
- But what really happens when you type an instruction like age = 42 in a Python module?
  - So, what happens is that an object is created. It gets an id, the type is set to int(integer number), and the value to 42.
  - A name, age, is placed in the global namespace, pointing to that object.
  - Therefore, whenever we are in the global namespace, after the execution of that line, we can retrieve that object by simply accessing it through its name: age.
- Here is a screenshot of what it may look like……

Figure 2.1: A name pointing to an object

- So, for the rest of this chapter, whenever you read something such as name = some_value,
- Think of a name placed in the namespace that is tied to the scope in which the instruction was written, with a nice arrow pointing to an object that has an id, a type, and a value.

Mutable or immutable? That is the question:

- The first fundamental distinction that Python makes on data is about whether or not the value of an object can change.

- If the value can change, the object is called mutable, whereas if the value cannot change, the object is called immutable.

- It is very important that you understand the distinction between mutable and immutable because it affects the code you write; take this example.

```
>>> age = 42
>>> age
42
>>> age = 43  #A
>>> age
43
```

  - In the preceding code, on line #A, have we changed the value of age? Well, no. But now it's 43.
  - Yes, it's 43, but 42 was an integer number, of the type int, which is immutable.
  - So, what happened is really that on the first line, age is a name that is set to point to an int object, whose value is 42.
  - When we type age = 43, what happens is that another object is created, of the type int and value 43 (the id will be different), and the name age is set to point to it.
  - So, in fact, we did not change that 42 to 43—we actually just pointed age to a different location, which is the new int object whose value is 43.

- Let's see the same code also printing the IDs:

```
>>> age = 42
>>> id(age)
4377553168
>>> age = 43
>>> id(age)
4377553200
```

- Notice that we print the IDs by calling the built-in id() function.
- As you can see, they are different, as expected. Bear in mind that age points to one object at a time: 42 first, then 43—never together.
- Now, let's see the same example using a mutable object. For this example, let's just use a Person object, that has a property age.
  - In this case, we set up an object fab whose type is Person (a custom class).
  - On creation, the object is given the age of 42. We then print it, along with the object ID, and the ID of age as well.
  - Notice that, even after we change age to be 25, the ID of fab stays the same (while the ID of age has changed, of course).

```
>>> class Person:
...     def __init__(self, age):
...         self.age = age
...
>>> fab = Person(age=42)
>>> fab.age
42
>>> id(fab)
4380878496
>>> id(fab.age)
4377553168
>>> fab.age = 25  # I wish!
>>> id(fab)  # will be the same
4380878496
>>> id(fab.age)  # will be different
4377552624
```

- Custom objects in Python are mutable (unless you code them not to be). Keep this concept in mind, as it's very important.

Numbers:

- Let's start by exploring Python's built-in data types for numbers.
- Python was designed by a man with a master's degree in mathematics and computer science, so it's only logical that it has amazing support for numbers.
- Numbers are immutable objects.

a) Integers:

- Python integers have an unlimited range, subject only to the available virtual memory.
- This means that it doesn't really matter how big a number you want to store is—as long as it can fit in your computer's memory, Python will take care of it.
- Integer numbers can be positive, negative, or 0 (zero).
- They support all the basic mathematical operations, as shown in the following example……

```
>>> a = 14
>>> b = 3
>>> a + b  # addition
17
>>> a - b  # subtraction
11
>>> a * b  # multiplication
42
>>> a / b  # true division
4.666666666666667
>>> a // b  # integer division
4
>>> a % b  # modulo operation (reminder of division)
2
>>> a ** b  # power operation
2744
```

- The preceding code should be easy to understand. Just notice one important thing:
  - Python has two division operators, one performs the so-called true division (/), which returns the quotient of the operands, and another one, the so-called integer division (//), which returns the floored quotient of the operands.
- It's worth noting that the power operator, **, also has a built-in function counterpart, pow(), shown in the example below……

```
>>> pow(10, 3)
1000.0  # result is float
>>> 10 ** 3
1000  # result is int
>>> pow(10, -3)
0.001
>>> 10 ** -3
0.001
```

- There is also an operator to calculate the remainder of a division. It's called the modulo operator, and it's represented by a percentage symbol (%).

```
>>> 10 % 3  # remainder of the division 10 // 3
1
>>> 10 % 4  # remainder of the division 10 // 4
2
```

- The pow() function allows a third argument to perform modular exponentiation.
- The form with three arguments now accepts a negative exponent in the case where the base is relatively prime to the modulus.
- The result is the modular multiplicative inverse of the base (or a suitable power of that, when the exponent is negative, but not -1), modulo the third argument. Here's an example……

```
>>> pow(123, 4)
228886641
>>> pow(123, 4, 100)
41  # notice: 228886641 % 100 == 41
>>> pow(37, -1, 43)  # modular inverse of 37 mod 43
7
>>> 7 * 37 % 43  # proof the above is correct
1
```

b) Booleans:

- Boolean algebra is that subset of algebra in which the values of the variables are the truth values, true and false.
- In Python, True and False are two keywords that are used to represent truth values.
- Booleans are a subclass of integers, so True and False behave respectively like 1 and 0.

- The equivalent of the int class for Booleans is the bool class, which returns either True or False.
- Every built-in Python object has a value in the Boolean context, which means they basically evaluate to either True or False when fed to the bool function.
- Boolean values can be combined in Boolean expressions using the logical operators and, or, and not. for simple example……

```
>>> int(True)  # True behaves like 1
1
>>> int(False)  # False behaves like 0
0
>>> bool(1)  # 1 evaluates to True in a Boolean context
True
>>> bool(-42)  # and so does every non-zero number
True
>>> bool(0)  # 0 evaluates to False
False
>>> # quick peek at the operators (and, or, not)
>>> not True
False
>>> not False
True
>>> True and True
True
>>> False or True
True
```

c) Real numbers:

- Real numbers, or floating-point numbers, are represented in Python according to the IEEE 754 double-precision binary floating point format.
- It is stored in 64 bits of information divided into three sections: sign, exponent, and mantissa.
- Several programming languages give coders two different formats: single and double precision. The former takes up 32 bits of memory, the latter 64.
- Python supports only the double format. Let's see a simple example……

```
>>> pi = 3.1415926536  # how many digits of PI can you remember?
>>> radius = 4.5
>>> area = pi * (radius ** 2)
>>> area
63.617251235400005
```

d) Complex numbers:

- Python gives you complex numbers support out of the box.
- Complex numbers are, they are numbers that can be expressed in the form a + ib, where a and b are real numbers, and i (or j if you're an engineer) is the imaginary unit; that is, the square root of -1. a and b are called, respectively, the real and imaginary part of the number.

```
>>> c = 3.14 + 2.73j
>>> c = complex(3.14, 2.73)  # same as above
>>> c.real  # real part
3.14
>>> c.imag  # imaginary part
2.73
>>> c.conjugate()  # conjugate of A + Bj is A - Bj
(3.14-2.73j)
>>> c * 2  # multiplication is allowed
(6.28+5.46j)
>>> c ** 2  # power operation as well
(2.4067000000000007+17.1444j)
>>> d = 1 + 1j  # addition and subtraction as well
>>> c - d
(2.14+1.73j)
```

e) Fractions and decimals:

- Let's finish the tour of the number department with a look at fractions and decimals.
- Fractions hold a rational numerator and denominator in their lowest forms. Let's see a quick example……

```
>>> from fractions import Fraction
>>> Fraction(10, 6)  # mad hatter?

Fraction(5, 3)  # notice it's been simplified
>>> Fraction(1, 3) + Fraction(2, 3)  # 1/3 + 2/3 == 3/3 == 1/1
Fraction(1, 1)
>>> f = Fraction(10, 6)
>>> f.numerator
5
>>> f.denominator
3
>>> f.as_integer_ratio()
(5, 3)
```

- Although Fraction objects can be very useful at times, it's not that common to spot them in commercial software.
- Instead, it is much more common to see decimal numbers being used in all those contexts where precision is everything; for example, in scientific and financial calculations.

```
>>> from decimal import Decimal as D  # rename for brevity
>>> D(3.14)  # pi, from float, so approximation issues
Decimal('3.140000000000000124344978758017532527446746826171875')
>>> D('3.14')  # pi, from a string, so no approximation issues
Decimal('3.14')
>>> D(0.1) * D(3) - D(0.3)  # from float, we still have the issue
Decimal('2.77555756156515654042363166 8E-17')
>>> D('0.1') * D(3) - D('0.3')  # from string, all perfect
Decimal('0.0')
>>> D('1.4').as_integer_ratio()  # 7/5 = 1.4 (isn't this cool?!)
(7, 5)
```

Immutable sequences:

- Let's start with immutable sequences: strings, tuples, and bytes.

1. Strings and bytes:

- Textual data in Python is handled with str objects, more commonly known as strings. They are immutable sequences of Unicode code points.
- Unicode code points can represent a character, but can also have other meanings, such as when formatting, for example.
- Python, unlike other languages, doesn't have a char type, so a single character is rendered simply by a string of length 1.
- Unicode is an excellent way to handle data, and should be used for the internals of any application.
- When it comes to storing textual data though, or sending it on the network, you will likely want to encode it, using an appropriate encoding for the medium you are using.
- The result of an encoding produces a bytes object, whose syntax and behavior is similar to that of strings.
- String literals are written in Python using single, double, or triple quotes (both single or double).
- If built with triple quotes, a string can span multiple lines. An example will clarify this……

```
>>> # 4 ways to make a string
>>> str1 = 'This is a string. We built it with single quotes.'
>>> str2 = "This is also a string, but built with double quotes."
>>> str3 = '''This is built using triple quotes,
... so it can span multiple lines.'''
>>> str4 = """This too
... is a multiline one
... built with triple double-quotes."""
>>> str4  #A
'This too\nis a multiline one\nbuilt with triple double-quotes.'
>>> print(str4)  #B
This too
is a multiline one
built with triple double-quotes.
```

- Strings, like any sequence, have a length. You can get this by calling the len() function.
- Python 3.9 has introduced two new methods that deal with the prefixes and suffixes of strings. Here's an example that explains the way they work……

```
>>> len(str1)
49

>>> s = 'Hello There'
>>> s.removeprefix('Hell')
'o There'
```

```
>>> s.removesuffix('here')
'Hello T'
>>> s.removeprefix('Ooops')
'Hello There'
```

Encoding and decoding strings:

- Using the encode/decode methods, we can encode Unicode strings and decode bytes objects.
- UTF-8 is a variable-length character encoding, capable of encoding all possible Unicode code points. It is the most widely used encoding for the web.
- Notice also that by adding the literal b in front of a string declaration, we're creating a bytes object.

```
>>> s = "This is üŋíc0de"  # unicode string: code points
>>> type(s)
<class 'str'>
>>> encoded_s = s.encode('utf-8')  # utf-8 encoded version of s
>>> encoded_s
b'This is \xc3\xbc\xc5\x8b\xc3\xadc0de'  # result: bytes object
>>> type(encoded_s)  # another way to verify it
<class 'bytes'>
>>> encoded_s.decode('utf-8')  # let's revert to the original
'This is üŋíc0de'
>>> bytes_obj = b"A bytes object"  # a bytes object
>>> type(bytes_obj)
<class 'bytes'>
```

Indexing and slicing strings:

- When manipulating sequences, it's very common to access them at one precise position (indexing), or to get a sub-sequence out of them (slicing).
- When dealing with immutable sequences, both operations are read-only.
  - While indexing comes in one form—zero-based access to any position within the sequence—slicing comes in different forms.
  - When you get a slice of a sequence, you can specify the start and stop positions, along with the step.
- They are separated with a colon (:) like this: my_sequence[start:stop:step]. All the arguments are optional; start is inclusive, and stop is exclusive.
- It's probably better to see an example, rather than try to explain them any further with words……
  - The last line is quite interesting. If you don't specify any of the parameters, Python will fill in the defaults for you.

```
>>> s = "The trouble is you think you have time."
>>> s[0]  # indexing at position 0, which is the first char
'T'
>>> s[5]  # indexing at position 5, which is the sixth char
'r'
>>> s[:4]  # slicing, we specify only the stop position
'The '
>>> s[4:]  # slicing, we specify only the start position
'trouble is you think you have time.'
```

```
>>> s[2:14]  # slicing, both start and stop positions
'e trouble is'
>>> s[2:14:3]  # slicing, start, stop and step (every 3 chars)
'erb '
>>> s[:]  # quick way of making a copy
'The trouble is you think you have time.'
```

String formatting:

- One of the features strings have is the ability to be used as a template. There are several different ways of formatting a string.

```
>>> greet_old = 'Hello %s!'
>>> greet_old % 'Fabrizio'
'Hello Fabrizio!'
>>> greet_positional = 'Hello {}!'
>>> greet_positional.format('Fabrizio')
'Hello Fabrizio!'
>>> greet_positional = 'Hello {} {}!'
>>> greet_positional.format('Fabrizio', 'Romano')
'Hello Fabrizio Romano!'
>>> greet_positional_idx = 'This is {0}! {1} loves {0}!'
>>> greet_positional_idx.format('Python', 'Heinrich')
'This is Python! Heinrich loves Python!'
>>> greet_positional_idx.format('Coffee', 'Fab')
'This is Coffee! Fab loves Coffee!'
>>> keyword = 'Hello, my name is {name} {last_name}'
>>> keyword.format(name='Fabrizio', last_name='Romano')
'Hello, my name is Fabrizio Romano'
```

- In the previous example, you can see four different ways of formatting strings……
  - The first one, which relies on the % operator, is deprecated and shouldn't be used anymore.
  - The current, modern way to format a string is by using the format() string method.
  - You can see, from the different examples, that a pair of curly braces acts as a placeholder within the string.
  - When we call format(), we feed it data that replaces the placeholders.
  - We can specify indexes (and much more) within the curly braces, and even names, which implies we'll have to call format() using keyword arguments instead of positional ones.
- One last feature we want to show you was added to Python in version 3.6, and it's called formatted string literals.
- This feature is quite cool (and it is faster than using the format() method): strings are prefixed with f, and contain replacement fields surrounded by curly braces.
- Replacement fields are expressions evaluated at runtime, and then formatted using the format protocol.

```
>>> name = 'Fab'
>>> age = 42
>>> f"Hello! My name is {name} and I'm {age}"
"Hello! My name is Fab and I'm 42"
>>> from math import pi
>>> f"No arguing with {pi}, it's irrational..."
"No arguing with 3.141592653589793, it's irrational..."
```

- An interesting addition to f-strings, which was introduced in Python 3.8, is the ability to add an equals sign specifier within the f-string clause;
  - This causes the expression to expand to the text of the expression, an equals sign, then the representation of the evaluated expression.
  - This is great for self-documenting and debugging purposes. Here's an example that shows the difference in behavior……

```
>>> user = 'heinrich'
>>> password = 'super-secret'
>>> f"Log in with: {user} and {password}"
'Log in with: heinrich and super-secret'
>>> f"Log in with: {user=} and {password=}"
"Log in with: user='heinrich' and password='super-secret'"
```

2. Tuples:

- A tuple is a sequence of arbitrary Python objects. In a tuple declaration, items are separated by commas.
- Tuples are used everywhere in Python. They allow for patterns that are quite hard to reproduce in other languages.
- Sometimes tuples are used implicitly;
  - For example, to set up multiple variables on one line, or
  - To allow a function to return multiple objects, and
  - In the Python console, tuples can be used implicitly to print multiple elements with one single instruction.
- We'll see examples for all these cases……

```
>>> t = ()  # empty tuple
>>> type(t)
<class 'tuple'>
>>> one_element_tuple = (42, )  # you need the comma!
>>> three_elements_tuple = (1, 3, 5)  # braces are optional here

>>> a, b, c = 1, 2, 3  # tuple for multiple assignment
>>> a, b, c  # implicit tuple to print with one instruction
(1, 2, 3)
>>> 3 in three_elements_tuple  # membership test
True
```

- Notice that the membership operator in can also be used with lists, strings, dictionaries, and, in general, with collection and sequence objects.
- One thing that tuple assignment allows us to do is one-line swaps, with no need for a third temporary variable.
- Because they are immutable, tuples can be used as keys for dictionaries.

Mutable sequences:

- Mutable sequences differ from their immutable counterparts in that they can be changed after creation. There are two mutable sequence types in Python: lists and byte arrays.

1) Lists:

- Python lists are very similar to tuples, but they don't have the restrictions of immutability.
- Lists are commonly used for storing collections of homogeneous objects, but there is nothing preventing you from storing heterogeneous collections as well.
- Lists can be created in many different ways. Let's see an example……

```
>>> []  # empty list
[]
>>> list()  # same as []
[]
>>> [1, 2, 3]  # as with tuples, items are comma separated
[1, 2, 3]
>>> [x + 5 for x in [2, 3, 4]]  # Python is magic
[7, 8, 9]
>>> list((1, 3, 5, 7, 9))  # list from a tuple
[1, 3, 5, 7, 9]
>>> list('hello')  # list from a string
['h', 'e', 'l', 'l', 'o']
```

- In the previous example, we showed you how to create a list using various techniques.
- We would like you to take a good look at the line with the comment Python is magic, is called a list comprehension. Let's see the main methods……

```
>>> a = [1, 2, 1, 3]
>>> a.append(13)  # we can append anything at the end
>>> a
[1, 2, 1, 3, 13]
>>> a.count(1)  # how many `1s` are there in the list?
2
>>> a.extend([5, 7])  # extend the list by another (or sequence)
>>> a
[1, 2, 1, 3, 13, 5, 7]
>>> a.index(13)  # position of `13` in the list (0-based indexing)
4
>>> a.insert(0, 17)  # insert `17` at position 0
>>> a
[17, 1, 2, 1, 3, 13, 5, 7]
>>> a.pop()  # pop (remove and return) last element
7
```

```
>>> a
[17, 1, 2, 3, 13, 5]
>>> a.remove(17)  # remove `17` from the list
>>> a
[1, 2, 3, 13, 5]
>>> a.reverse()  # reverse the order of the elements in the list
>>> a
[5, 13, 3, 2, 1]
>>> a.sort()  # sort the list
>>> a
[1, 2, 3, 5, 13]
>>> a.clear()  # remove all elements from the list
>>> a
[]
```

- The preceding code gives you a roundup of a list's main methods. We want to show you how powerful they are, using the method extend() as an example……

```
>>> a = list('hello')  # makes a list from a string
>>> a
['h', 'e', 'l', 'l', 'o']
>>> a.append(100)  # append 100, heterogeneous type
>>> a
['h', 'e', 'l', 'l', 'o', 100]
>>> a.extend((1, 2, 3))  # extend using tuple
>>> a
['h', 'e', 'l', 'l', 'o', 100, 1, 2, 3]
>>> a.extend('...')  # extend using string
>>> a
['h', 'e', 'l', 'l', 'o', 100, 1, 2, 3, '.', '.', '.']
```

- Now, let's see the most common operations you can do with lists. Notice how easily we can perform the sum and the product of all values in a list.

```
>>> a = [1, 3, 5, 7]
>>> min(a)  # minimum value in the list
1
>>> max(a)  # maximum value in the list
7
>>> sum(a)  # sum of all values in the list
16
>>> from math import prod
>>> prod(a)  # product of all values in the list
105
>>> len(a)  # number of elements in the list
4
>>> b = [6, 7, 8]
>>> a + b  # `+` with list means concatenation
[1, 3, 5, 7, 6, 7, 8]
>>> a * 2  # `*` has also a special meaning
[1, 3, 5, 7, 1, 3, 5, 7]
```

- The last two lines in the preceding code are also quite interesting, as they introduce us to a concept called operator overloading.

- In short, this means that operators, such as +, -, *, %, and so on, may represent different operations according to the context they are used in.

2) Bytearrays:

- To conclude our overview of mutable sequence types, let's spend a moment on the bytearray type. Basically, they represent the mutable version of bytes objects.
- They expose most of the usual methods of mutable sequences as well as most of the methods of the bytes type. Items in a bytearray are integers in the range [0, 256).
- Let's see an example with the bytearray type……

```
>>> bytearray()  # empty bytearray object
bytearray(b'')
```

```
>>> bytearray(10)  # zero-filled instance with given length
bytearray(b'\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00')
>>> bytearray(range(5))  # bytearray from iterable of integers
bytearray(b'\x00\x01\x02\x03\x04')
>>> name = bytearray(b'Lina')  #A - bytearray from bytes
>>> name.replace(b'L', b'l')
bytearray(b'lina')
>>> name.endswith(b'na')
True
>>> name.upper()
bytearray(b'LINA')
>>> name.count(b'L')
1
```

Set types:

- Python also provides two set types, **set and frozenset**. The set type is mutable, while frozenset is immutable.
- They are unordered collections of immutable objects.
- Hashability is a characteristic that allows an object to be used as a set member as well as a key for a dictionary.
- Objects that compare equally must have the same hash value. Sets are very commonly used to test for membership. let's introduce the **in** operator example……

```
>>> small_primes = set()  # empty set
>>> small_primes.add(2)  # adding one element at a time
>>> small_primes.add(3)
>>> small_primes.add(5)
>>> small_primes
{2, 3, 5}
>>> small_primes.add(1)  # Look what I've done, 1 is not a prime!
>>> small_primes
{1, 2, 3, 5}
>>> small_primes.remove(1)  # so let's remove it
```

```
>>> 3 in small_primes  # membership test
True
>>> 4 in small_primes
False
>>> 4 not in small_primes  # negated membership test
True
>>> small_primes.add(3)  # trying to add 3 again
```

```
>>> small_primes
{2, 3, 5}  # no change, duplication is not allowed
>>> bigger_primes = set([5, 7, 11, 13])  # faster creation
>>> small_primes | bigger_primes  # union operator `|`
{2, 3, 5, 7, 11, 13}
>>> small_primes & bigger_primes  # intersection operator `&`
{5}
>>> small_primes - bigger_primes  # difference operator `-`
{2, 3}
```

- In the preceding code, you can see two different ways to create a set.
  - One creates an empty set and then adds elements one at a time.
  - The other creates the set using a list of numbers as an argument to the constructor, which does all the work for us.
- Of course, you can create a set from a list or tuple (or any iterable) and then you can add and remove members from the set as you please.
- Another way of creating a set is by simply using the curly braces notation, like this……

```
>>> small_primes = {2, 3, 5, 5, 3}
>>> small_primes
{2, 3, 5}
```

- Notice we added some duplication to emphasize that the resulting set won't have any.
- Let's see an example using the immutable counterpart of the set type, frozenset.
- As you can see, frozenset objects are quite limited with respect to their mutable counterpart.

```
>>> small_primes = frozenset([2, 3, 5, 7])
>>> bigger_primes = frozenset([5, 7, 11])
>>> small_primes.add(11)  # we cannot add to a frozenset
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'frozenset' object has no attribute 'add'
>>> small_primes.remove(2)  # nor can we remove
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'frozenset' object has no attribute 'remove'
>>> small_primes & bigger_primes  # intersect, union, etc. allowed
frozenset({5, 7})
```

- They still prove very effective for membership test, union, intersection, and difference operations, and for performance reasons.

Mapping types: dictionaries:

- Of all the built-in Python data types, the dictionary is easily the most interesting.
- It's the only standard mapping type, and it is the backbone of every Python object.
- A dictionary maps keys to values.
  - Keys need to be hashable objects, while values can be of any arbitrary type. Dictionaries are also mutable objects.
- There are quite a few different ways to create a dictionary,
- So let us give you a simple example of how to create a dictionary equal to {'A': 1, 'Z': -1} in five different ways……

```
>>> a = dict(A=1, Z=-1)
>>> b = {'A': 1, 'Z': -1}
>>> c = dict(zip(['A', 'Z'], [1, -1]))
>>> d = dict([('A', 1), ('Z', -1)])
>>> e = dict({'Z': -1, 'A': 1})
>>> a == b == c == d == e  # are they all the same?
True  # They are indeed
```

- While to check whether an object is the same as another one (or five in one go, in this case), we use double equals.
- There is also another way to compare objects, which involves the is operator, and checks whether the two objects are the same (that is, that they have the same ID, not just the same value).
- In the preceding code, we also used one nice function: zip().
- It is named after the real-life zip, which glues together two parts, taking one element from each part at a time.

```
>>> list(zip(['h', 'e', 'l', 'l', 'o'], [1, 2, 3, 4, 5]))
[('h', 1), ('e', 2), ('l', 3), ('l', 4), ('o', 5)]
>>> list(zip('hello', range(1, 6)))  # equivalent, more pythonic
[('h', 1), ('e', 2), ('l', 3), ('l', 4), ('o', 5)]
```

- Let's go back to dictionaries and see how many wonderful methods they expose for allowing us to manipulate them as we want.
- Let's start with the basic operations.
  - Notice how accessing keys of a dictionary, regardless of the type of operation we're performing, is done using square brackets.

```
>>> d = {}
>>> d['a'] = 1  # let's set a couple of (key, value) pairs
>>> d['b'] = 2
>>> len(d)  # how many pairs?
2
>>> d['a']  # what is the value of 'a'?
1
>>> d  # how does `d` look now?
{'a': 1, 'b': 2}
>>> del d['a']  # let's remove `a`
>>> d
{'b': 2}
>>> d['c'] = 3  # let's add 'c': 3
>>> 'c' in d  # membership is checked against the keys
True
>>> 3 in d  # not the values
False
>>> 'e' in d
False
>>> d.clear()  # let's clean everything from this dictionary
>>> d
{}
```

- Let's now look at three special objects called dictionary views: keys, values, and items.
- These objects provide a dynamic view of the dictionary entries and they change when the dictionary changes.
  - **keys()** returns all the keys in the dictionary,
  - **values()** returns all the values in the dictionary, and
  - **items()** returns all the (key, value) pairs in the dictionary.

```
>>> d = dict(zip('hello', range(5)))
>>> d
{'h': 0, 'e': 1, 'l': 3, 'o': 4}
>>> d.keys()
dict_keys(['h', 'e', 'l', 'o'])
>>> d.values()
dict_values([0, 1, 3, 4])
>>> d.items()
dict_items([('h', 0), ('e', 1), ('l', 3), ('o', 4)])
>>> 3 in d.values()
True
>>> ('o', 4) in d.items()
True
```

- Let's take a look now at some other methods exposed by Python's dictionaries……

```
>>> d
{'h': 0, 'e': 1, 'l': 3, 'o': 4}
>>> d.popitem()  # removes a random item (useful in algorithms)
('o', 4)
>>> d
{'h': 0, 'e': 1, 'l': 3}
>>> d.pop('l')  # remove item with key `l`
3
```

```
>>> d.pop('not-a-key')  # remove a key not in dictionary: KeyError
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'not-a-key'
>>> d.pop('not-a-key', 'default-value')  # with a default value?
'default-value'  # we get the default value
>>> d.update({'another': 'value'})  # we can update dict this way
>>> d.update(a=13)  # or this way (like a function call)
>>> d
{'h': 0, 'e': 1, 'another': 'value', 'a': 13}
>>> d.get('a')  # same as d['a'] but if key is missing no KeyError
13
>>> d.get('a', 177)  # default value used if key is missing
13
>>> d.get('b', 177)  # like in this case
177
>>> d.get('b')  # key is not there, so None is returned
```

- One last method we really like about dictionaries is setdefault(). It behaves like get(), but also sets the key with the given value if it is not there.

```
>>> d = {}
>>> d.setdefault('a', 1)  # 'a' is missing, we get default value
1
>>> d
{'a': 1}  # also, the key/value pair ('a', 1) has now been added
>>> d.setdefault('a', 5)  # let's try to override the value
1
>>> d
{'a': 1}  # no override, as expected
```

The collections module:

- When Python general-purpose built-in containers (tuple, list, set, and dict) aren't enough, we can find specialized container data types in the collection's module. They are described in Table 2.1

Data type	Description
namedtuple()	Factory function for creating tuple subclasses with named fields
deque	List-like container with fast appends and pops on either end
ChainMap	Dictionary-like class for creating a single view of multiple mappings
Counter	Dictionary subclass for counting hashable objects
OrderedDict	Dictionary subclass with methods that allow for re-ordering entries
defaultdict	Dictionary subclass that calls a factory function to supply missing values
UserDict	Wrapper around dictionary objects for easier dictionary subclassing
UserList	Wrapper around list objects for easier list subclassing
UserString	Wrapper around string objects for easier string subclassing

Table 2.1: Collections module data types

a. Namedtuple:

- A namedtuple is a tuple-like object that has fields accessible by attribute lookup, as well as being indexable and iterable (it's actually a subclass of tuple).

- This is sort of a compromise between a fully-fledged object and a tuple, and it can be useful in those cases where you don't need the full power of a custom object, but only want your code to be more readable by avoiding weird indexing.
- Another use case is when there is a chance that items in the tuple need to change their position after refactoring, forcing the coder to also refactor all the logic involved, which can be very tricky.

- For example, say we are handling data about the left and right eyes of a patient.
  - We save one value for the left eye (position 0) and one for the right eye (position 1) in a regular tuple. Here's how that may look……
  - Now let's pretend we handle vision objects all of the time, and the designer decides to enhance them by adding information for the combined vision, so that a vision (left eye, combined, right eye).

```
>>> vision = (9.5, 8.8)
>>> vision
(9.5, 8.8)
>>> vision[0]  # left eye (implicit positional reference)
9.5
>>> vision[1]  # right eye (implicit positional reference)
8.8
```

- We may have a lot of code that depends on vision[0] being the left eye information and vision[1] being the right eye information. We have to restructure our code wherever we handle these objects.
- We could have probably approached this a bit better from the beginning, by using a namedtuple.
- If, within our code, we refer to the left and right eyes using vision.left and vision.right, all we need to do to fix the new design issue is change our factory and the way we create instances.

```
>>> from collections import namedtuple
>>> Vision = namedtuple('Vision', ['left', 'right'])
>>> vision = Vision(9.5, 8.8)
>>> vision[0]
9.5
>>> vision.left  # same as vision[0], but explicit
9.5
>>> vision.right  # same as vision[1], but explicit
8.8
>>> Vision = namedtuple('Vision', ['left', 'combined', 'right'])
>>> vision = Vision(9.5, 9.2, 8.8)
>>> vision.left  # still correct
9.5
>>> vision.right  # still correct (though now is vision[2])
8.8
>>> vision.combined  # the new vision[1]
9.2
```

- You can see how convenient it is to refer to those values by name rather than by position.

b. Defaultdict:

- The defaultdict data type is one of our favorites.
- It allows you to avoid checking whether a key is in a dictionary by simply inserting it for you on your first access attempt, with a default value whose type you pass on creation.
- In some cases, this tool can be very handy and shorten your code a little. Let's see a quick example……
- Say we are updating the value of age, by adding one year. If age is not there, we assume it was 0 and we update it to 1.

```
>>> d = {}
>>> d['age'] = d.get('age', 0) + 1  # age not there, we get 0 + 1
>>> d
{'age': 1}
>>> d = {'age': 39}
>>> d['age'] = d.get('age', 0) + 1  # age is there, we get 40
>>> d
{'age': 40}
```

- Now let's see how it would work with a defaultdict data type……

```
>>> from collections import defaultdict
>>> dd = defaultdict(int)  # int is the default type (0 the value)
>>> dd['age'] += 1  # short for dd['age'] = dd['age'] + 1
>>> dd
defaultdict(<class 'int'>, {'age': 1})  # 1, as expected
```

- Notice how we just need to instruct the defaultdict factory that we want an int number to be used if the key is missing (we'll get 0, which is the default for the int type).

c. ChainMap:

- ChainMap is an extremely useful data type which was introduced in Python 3.3.
- It behaves like a normal dictionary but, according to the Python documentation, is provided for quickly linking a number of mappings so they can be treated as a single unit.
- This is usually much faster than creating one dictionary and running multiple update calls on it.
- ChainMap can be used to simulate nested scopes and is useful in templating.
- The underlying mappings are stored in a list. That list is public and can be accessed or updated using the maps attribute.
- Lookups search the underlying mappings successively until a key is found. By contrast, writes, updates, and deletions only operate on the first mapping.

- A very common use case is providing defaults, so let's see an example……

```
>>> from collections import ChainMap
>>> default_connection = {'host': 'localhost', 'port': 4567}
>>> connection = {'port': 5678}
>>> conn = ChainMap(connection, default_connection) # map creation
>>> conn['port']  # port is found in the first dictionary
5678
>>> conn['host']  # host is fetched from the second dictionary
'localhost'
>>> conn.maps  # we can see the mapping objects
[{'port': 5678}, {'host': 'localhost', 'port': 4567}]
>>> conn['host'] = 'packtpub.com'  # let's add host
>>> conn.maps
[{'port': 5678, 'host': 'packtpub.com'},
 {'host': 'localhost', 'port': 4567}]
>>> del conn['port']  # let's remove the port information
>>> conn.maps
[{'host': 'packtpub.com'}, {'host': 'localhost', 'port': 4567}]
>>> conn['port']  # now port is fetched from the second dictionary
4567
>>> dict(conn)  # easy to merge and convert to regular dictionary
{'host': 'packtpub.com', 'port': 4567}
```

- You work on a ChainMap object, configure the first mapping as you want, and when you need a complete dictionary with all the defaults as well as the customized items, you can just feed the ChainMap object to a dict constructor.

Enums:

- Technically not a built-in data type, as you have to import them from the enum module, but definitely worth mentioning, are enumerations.
- They were introduced in Python 3.4, and though it is not that common to see them in professional code.
- Say you need to represent traffic lights; in your code, you might resort to the following……

```
>>> GREEN = 1
>>> YELLOW = 2
>>> RED = 4
>>> TRAFFIC_LIGHTS = (GREEN, YELLOW, RED)
>>> # or with a dict
>>> traffic_lights = {'GREEN': 1, 'YELLOW': 2, 'RED': 4}
```

```
>>> from enum import Enum
>>> class TrafficLight(Enum):
...     GREEN = 1
...     YELLOW = 2
...     RED = 4
...
```

```
>>> TrafficLight.GREEN
<TrafficLight.GREEN: 1>
>>> TrafficLight.GREEN.name
'GREEN'
>>> TrafficLight.GREEN.value
1
>>> TrafficLight(1)
<TrafficLight.GREEN: 1>
>>> TrafficLight(4)
<TrafficLight.RED: 4>
```

Conditionals and Iteration:

- In order to control the flow of a program, we have two main weapons: conditional programming (also known as branching) and looping. We can use them in many different combinations and variations.

Conditional programming:

- Conditional programming, or branching, is something you do every day, every moment.
  - It's about evaluating conditions: if the light is green, then I can cross; if it's raining, then I'm taking the umbrella; and if I'm late for work, then I'll call my manager.
- The main tool is the if statement, which comes in different forms and colors.
- Its basic function is to evaluate an expression and, based on the result, choose which part of the code to execute. As usual, let's look at an example.

```
# conditional.1.py
late = True
if late:
    print('I need to call my manager!')
```

- This is possibly the simplest example: when fed to the if statement, late acts as a conditional expression, which is evaluated in a Boolean context.
  - If the result of the evaluation is True, then we enter the body of the code immediately after the if statement.
- Notice that the print instruction is indented, which means that it belongs to a scope defined by the if clause.

```
# conditional.2.py
late = False
if late:
    print('I need to call my manager!')  #1
else:
    print('no need to call my manager...')  #2
```

- Depending on the result of evaluating the late expression, we can either enter block #1 or block #2, but not both.
- The preceding example also introduces the else clause, which becomes very handy when we want to provide an alternative set of instructions to be executed when an expression evaluates to False within an if clause. The else clause is optional.

❖ **A specialized else: elif:**

- Sometimes all you need is to do something if a condition is met (a simple if clause). At other times, you need to provide an alternative, in case the condition is False (if/ else clause).
- But there are situations where you may have more than two paths to choose from…

```
# taxes.py
income = 15000
if income < 10000:
    tax_coefficient = 0.0  #1
elif income < 30000:
    tax_coefficient = 0.2  #2
elif income < 100000:
    tax_coefficient = 0.35  #3
else:
    tax_coefficient = 0.45  #4

print(f'You will pay: ${income * tax_coefficient} in taxes')
```

- Let's now see another example that shows us how to nest if clauses. Say your program encounters an error.
  - If the alert system is the console, we print the error.
  - If the alert system is an email, we send it according to the severity of the error.
  - If the alert system is anything other than console or email, we don't know what to do, therefore we do nothing. Let's put this into code……

```
# errorsalert.py
alert_system = 'console'  # other value can be 'email'
error_severity = 'critical'  # other values: 'medium' or 'low'
error_message = 'OMG! Something terrible happened!'

if alert_system == 'console':
    print(error_message)  #1
elif alert_system == 'email':
    if error_severity == 'critical':
        send_email('admin@example.com', error_message)  #2
    elif error_severity == 'medium':
        send_email('support.1@example.com', error_message)  #3
    else:
        send_email('support.2@example.com', error_message)  #4
```

❖ **The ternary operator:**

- The ternary operator or, in layman's terms, the short version of an if/else clause.
- When the value of a name is to be assigned according to some condition, it is sometimes easier and more readable to use the ternary operator instead of a proper if clause. For example, instead of……

```
# ternary.py
order_total = 247  # GBP

# classic if/else form

if order_total > 100:
    discount = 25  # GBP
else:
    discount = 0  # GBP
print(order_total, discount)
```

- We could write……

```
# ternary.py
# ternary operator
discount = 25 if order_total > 100 else 0
print(order_total, discount)
```

Looping:

- Looping means being able to repeat the execution of a code block more than once, according to the loop parameters given.
- There are different looping constructs, which serve different purposes, and Python has distilled all of them down to just two, which you can use to achieve everything you need. These are the for and while statements.

The for loop:

- The for loop is used when looping over a sequence, such as a list, tuple, or collection of objects.
- Let's start with a simple example and expand on the concept to see what the Python syntax allows us to do.

```
# simple.for.py
for number in [0, 1, 2, 3, 4]:
    print(number)
```

  - This simple snippet of code, when executed, prints all numbers from 0 to 4.
  - The for loop is fed the list [0, 1, 2, 3, 4] and, at each iteration number, is given a value from the sequence, then the body of the loop is executed (the print() line).

- When the sequence is exhausted, the for loop terminates, and the execution of the code resumes normally with the code after the loop.

a) Iterating over a range:

- Sometimes we need to iterate over a range of numbers, and it would be quite unpleasant to have to do so by hardcoding the list somewhere.
- In such cases, the range() function comes to the rescue.

```
# simple.for.py
for number in range(5):
    print(number)
```

- The range() function is used extensively in Python programs when it comes to creating sequences.
- You can call it by passing one value, which acts as stop (counting from 0), or you can pass two values (start and stop), or even three (start, stop, and step).

```
>>> list(range(10))  # one value: from 0 to value (excluded)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> list(range(3, 8))  # two values: from start to stop (excluded)
[3, 4, 5, 6, 7]
>>> list(range(-10, 10, 4))  # three values: step is added
[-10, -6, -2, 2, 6]
```

b) Iterating over a sequence:

- We now have all the tools to iterate over a sequence, so let's build on that example……

```
# simple.for.2.py
surnames = ['Rivest', 'Shamir', 'Adleman']
for position in range(len(surnames)):
    print(position, surnames[position])
```

- Now, this style of looping is actually much closer to languages such as Java or C.
- In Python, you can just iterate over any sequence or collection, so there is no need to get the list of positions and retrieve elements out of a sequence at each iteration.
- Let's change the example into a more Pythonic form……

```
# simple.for.3.py
surnames = ['Rivest', 'Shamir', 'Adleman']
for surname in surnames:
    print(surname)
```

- The for loop can iterate over the surnames list, and it gives back each element in order at each iteration.
- What if you wanted to print the position as well, though? Or what if you actually needed it?
- Should you go back to the range(len(...)) form? No. You can use the enumerate() built-in function, like this……

```
# simple.for.4.py
surnames = ['Rivest', 'Shamir', 'Adleman']
for position, surname in enumerate(surnames):
    print(position, surname)
```

  - Notice that enumerate() gives back a two-tuple (position, surname) at each iteration, but still, it's much more readable (and more efficient) than the range(len(...)) example.
  - You can call enumerate() with a start parameter, such as enumerate(iterable, start), and it will start from start, rather than 0.

c) Iterating over multiple sequences:

- Let's see another example of how to iterate over two sequences of the same length, in order to work on their respective elements in pairs.
- Say we have a list of people and a list of numbers representing the age of the people in the first list. We want to print the pair person/age on one line for each of them.
- Let's start with an example, which we will refine gradually……

```
# multiple.sequences.py
people = ['Nick', 'Rick', 'Roger', 'Syd']
ages = [23, 24, 23, 21]
for position in range(len(people)):
    person = people[position]
    age = ages[position]
    print(person, age)
```

- Wouldn't it be nice if we could use the same approach as for iterating over a single sequence? Let's try to improve it by using enumerate()……

```
# multiple.sequences.enumerate.py
people = ['Nick', 'Rick', 'Roger', 'Syd']
ages = [23, 24, 23, 21]
for position, person in enumerate(people):
    age = ages[position]
    print(person, age)
```

- That's better, but still not perfect. We're iterating properly on people, but we're still fetching age using positional indexing.

```
# multiple.sequences.zip.py
people = ['Nick', 'Rick', 'Roger', 'Syd']
ages = [23, 24, 23, 21]
for person, age in zip(people, ages):
    print(person, age)
```

- Let's expand a little on the previous example in two ways, using explicit and implicit assignment:

```
# multiple.sequences.explicit.py
people = ['Nick', 'Rick', 'Roger', 'Syd']
ages = [23, 24, 23, 21]
instruments = ['Drums', 'Keyboards', 'Bass', 'Guitar']
for person, age, instrument in zip(people, ages, instruments):
    print(person, age, instrument)
```

```
# multiple.sequences.implicit.py
people = ['Nick', 'Rick', 'Roger', 'Syd']
ages = [23, 24, 23, 21]
instruments = ['Drums', 'Keyboards', 'Bass', 'Guitar']
for data in zip(people, ages, instruments):
    person, age, instrument = data
    print(person, age, instrument)
```

While loop:

- The for loop is best suited in cases where you need to iterate over the elements of some container or other iterable object.
- There are other cases though, when you just need to loop until some condition is satisfied, or even loop indefinitely until the application is stopped, such as cases Python provides us with the while loop.
- The while loop is similar to the for loop in that they both loop, and at each iteration they execute a body of instructions.
- The difference is that the while loop doesn't loop over a sequence; rather, it loops as long as a certain condition is satisfied. When the condition is no longer satisfied, the loop ends.

Let's write some code to calculate the binary representation for the number 39, 100111_2:

```
# binary.py
n = 39
remainders = []
while n > 0:
    remainder = n % 2  # remainder of division by 2
    remainders.append(remainder)  # we keep track of remainders
    n //= 2  # we divide n by 2

remainders.reverse()
print(remainders)
```

- We can make the code a little shorter (and more Pythonic), by using the divmod() function, which is called with a number and a divisor, and returns a tuple with the result of the integer division and its remainder.
- For example, divmod(13, 5) would return (2, 3)

```
# binary.2.py
n = 39
remainders = []
while n > 0:
    n, remainder = divmod(n, 2)
    remainders.append(remainder)

remainders.reverse()
print(remainders)
```

- In the preceding code, we have reassigned n to the result of the division by 2, along with the remainder, in one single line.

The break and continue statements:

- According to the task at hand, sometimes you will need to alter the regular flow of a loop.
- You can either skip a single iteration (as many times as you want), or you can break out of the loop entirely.
- A common use case for skipping iterations is, for example, when you are iterating over a list of items and you need to work on each of them only if some condition is verified.
- On the other hand, if you're iterating over a collection of items, and you have found one of them that satisfies some need you have, you may decide not to continue the loop entirely and therefore break out of it.
- Let's say you want to apply a 20% discount to all products in a basket list for those that have an expiration date of today.
  - The way you achieve this is to use the continue statement……
  - Which tells the looping construct to stop execution of the body immediately and go to the next iteration, if any.

```
# discount.py
from datetime import date, timedelta

today = date.today()
tomorrow = today + timedelta(days=1)  # today + 1 day is tomorrow
products = [
    {'sku': '1', 'expiration_date': today, 'price': 100.0},
    {'sku': '2', 'expiration_date': tomorrow, 'price': 50},
    {'sku': '3', 'expiration_date': today, 'price': 20},
]
```

```
for product in products:
    if product['expiration_date'] != today:
        continue
    product['price'] *= 0.8  # equivalent to applying 20% discount
    print(
        'Price for sku', product['sku'],
        'is now', product['price'])
```

- Let's see an example of breaking out of a loop. Say we want to tell whether at least one of the elements in a list evaluates to True when fed to the bool() function.
- Given that we need to know whether there is at least one, when we find it, we don't need to keep scanning the list any further.
- In Python code, this translates to using the break statement.

Putting all this together:

- Now that you have seen all there is to see about conditionals and loops, it's time to spice things up a little.
- We'll mix and match here, so you can see how you can use all these concepts together.
- **Break example:**

```
# any.py
items = [0, None, 0.0, True, 0, 7]  # True and 7 evaluate to True

found = False  # this is called "flag"
for item in items:
    print('scanning item', item)
    if item:
        found = True  # we update the flag
        break

if found:  # we inspect the flag
    print('At least one item evaluates to True')
else:
    print('All items evaluate to False')
```

- Let's start by writing some code to generate a list of prime numbers up to some limit.

```
# primes.py
primes = []  # this will contain the primes at the end
upto = 100  # the limit, inclusive
for n in range(2, upto + 1):
    is_prime = True  # flag, new at each iteration of outer for
    for divisor in range(2, n):
        if n % divisor == 0:
            is_prime = False
            break
```

```
    if is_prime:  # check on flag
        primes.append(n)
print(primes)
```

A quick peek at the itertools module:

- A chapter about iterables, iterators, conditional logic, and looping wouldn't be complete without a few words about the itertools module.

a. Infinite iterators:

- Infinite iterators allow you to work with a for loop in a different fashion, such as if it were a while loop.

```
# infinite.py
from itertools import count

for n in count(5, 3):
    if n > 20:
        break
    print(n, end=', ') # instead of newline, comma and space
```

- The count factory class makes an iterator that simply goes on and on counting.
- It starts from 5 and keeps adding 3 to it. We need to break it manually if we don't want to get stuck in an infinite loop.

b. Iterators terminating on the shortest input sequence:

- It allows you to create an iterator based on multiple iterators, combining their values according to some logic.
- The key point here is that among those iterators, if any of them are shorter than the rest, the resulting iterator won't break, but will simply stop as soon as the shortest iterator is exhausted.
- Let us give you an example using compress().
  - This iterator gives you back the data according to a corresponding item in a selector being True or False;
  - compress('ABC', (1, 0, 1)) would give back 'A' and 'C', because they correspond to 1. Let's see a simple example.

```
# compress.py
from itertools import compress
data = range(10)
even_selector = [1, 0] * 10
odd_selector = [0, 1] * 10

even_numbers = list(compress(data, even_selector))
odd_numbers = list(compress(data, odd_selector))
```

```
print(odd_selector)
print(list(data))
print(even_numbers)
print(odd_numbers)
```

- Notice that odd_selector and even_selector are 20 elements in length, while data is only 10.
- compress() will stop as soon as data has yielded its last element. Running this code produces the following……

```
$ python compress.py
[0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1]
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
[0, 2, 4, 6, 8]
[1, 3, 5, 7, 9]
```

- It's a very fast and convenient way of selecting elements out of an iterable.
- Instead of using a for loop to iterate over each value that is given back by the compress() calls, we used list(), which does the same.

c. Combinatoric generators:

- Last but not least is combinatoric generators. These are really fun, if you are into this kind of thing. Let's look at a simple example on permutations.
  - For example, there are six permutations of ABC: ABC, ACB, BAC, BCA, CAB, and CBA.
  - If a set has N elements, then the number of permutations of them is N! (N factorial).
  - For the ABC string, the permutations are 3! = 3 * 2 * 1 = 6. Let's see this in Python……

```
# permutations.py
from itertools import permutations
print(list(permutations('ABC')))
```

References:

- Learn Python Programming Third Edition, Packt - Fabrizio Romano and Heinrich Kruger

SAMPLE QUESTIONS:

1. Python understands object very well. Discuss the above statement with main features of Objects.
2. Explain the different qualities exhibited by Python.

3. Explain the different components of Python's execution model.
4. Python object is Immutable or Mutable, Discuss its effect with an example.
5. Discuss the different built-in datatypes available to manipulate Numbers.
6. Explain the concept of Indexing and slicing with string with an example.
7. Discuss the different ways of formatting the strings with an example.
8. Explain the List as Mutable object in python with its constructors and methods.
9. Discuss the different type of set available in python with operators used with it.
10. Explain the Dictionaries as a mapping type with an example with methods available with it.
11. Explain briefly with an example:

    a. Namedtuple b. defaultdict

12. Explain the different use-cases of for loop in python using an example.
13. Differentiate the behavior of break and continue statements with an example.
14. Explain the different tools available in itertools module with its usage.

SAMPLE PROGRAMS:

1. Write a python program to check whether given number is a prime number or not.
2. Write a python program to calculate the nCr.
3. Write a python program to check whether given number is palindrome or not.
4. Write a python program to convert given Decimal number to its Binary equivalent.
5. Write a python program to accept ‘N’ values from user and append to a list, now display only the even values by reading the list.
6. Write a python program to accept ‘N’ values from user and append to a list. Now by traversing the list display each element along with its maximum digit.
7. Write a python program to accept ‘N’ names from user and append to a list. Now by traversing the list display only the names which contain vowels in it.

****